

NUMARA P11 Convergence Harness: Results Report

Internal experimental documentation for V0.1 deterministic sanity check and V0.2 stochastic robustness harness.

Prepared: 2026-07-08. Scope: local pre-production harness; not a production intelligence claim; not external scientific validation.

Executive Summary

This report documents the current NUMARA P11 test harness results. The project is testing whether different heuristic path fields can converge on the same structural invariant without a single hard-coded deterministic route. The specific invariant tested in the build-day transfer task is: an event is an output; the durable process survives the event; community memory carries continuity.

The V0.1 deterministic harness passed its sanity check. The V0.2 stochastic harness produced a more important result: the transfer invariant survived randomized perturbation above threshold, but the adversarial guard and false-convergence controls failed. This is a useful failure profile. It suggests the system has real convergence pressure, but the current graph is too eager to converge on the known invariant even under negative controls.

Claim	Current status	Evidence
P11 convergence exists in this harness	Supported internally	V0.1: 6/6 invariant count; V0.2: 81.9% invariant success under noise
The invariant transfers across task type	Supported internally	Grant logic generalized into build-day follow-up logic
The graph is robust enough for production	Not supported	V0.2 adversarial guard pass rate 0.0 and false-convergence rate 66.7%
Next step should be larger tests	Yes, after gating fixes	V0.3 must add task gates, negative controls, and guard-driven suppression

Research Question

Primary question: can different heuristic path stacks produce the same or similar invariant output structure while preserving primitive matching and guard boundaries?

Operational framing: P11 equivalence is considered active when multiple different activation paths converge on the same invariant structure without being forced through a single route. This is an internal convergence signal, not an external truth claim.

Divergence creates candidates.
Convergence creates confidence.
Guards decide what may publish.

System Under Test

The current construct is a layered pre-production harness. It is not a self-resolving production agent. It is a test scaffold for primitive-locked graph behavior.

Layer	Purpose
Primitive basis	13-dimensional canonical P0-P12 vector basis
Graph topology	Node and edge structure with typed supports, translates, inhibits, and guards
Heuristic path field	Biases groups of nodes without forcing a fixed route

Layer	Purpose
Guard rules	Suppress unsafe or context-mismatched output pathways
P11 scoring	Measures invariant convergence across different paths
Audit artifacts	Results, run logs, reports, and rollback manifests

Harness Versions

Version	Purpose	Status	Key result
V0.1	Deterministic sanity check	pass	All six paths produced event_output, durable_process, and community_memory
V0.2	Stochastic perturbation and guard tests	partial	Transfer survived noise; adversarial guard and false-convergence failed

Main Results

V0.1 Deterministic Sanity Check

Metric	Value
Status	pass
Primitive basis preserved	True
event_output invariant count	6/6
durable_process invariant count	6/6
community_memory invariant count	6/6
Mean structural convergence	0.925926
Mean primitive cosine	0.999699

Interpretation: V0.1 proved that the graph files, path definitions, primitive vectors, and deterministic runner were coherent enough to reproduce the invariant. It did not test noise, negative controls, or adversarial conditions.

V0.2 Stochastic Robustness Harness

Metric	Value
Status	partial
Total runs	192
Transfer trial count	144
Invariant success rate	81.9%
Unsafe failure rate	0.0%
Technical guard gap count	0
Mean structural convergence	0.797212
Mean primitive cosine	0.999026
Adversarial guard pass rate	0.0%
False convergence rate	66.7%

Interpretation: the transfer invariant survived perturbation in 81.9% of 144 transfer trials with zero unsafe failures and zero technical guard gaps. However, the adversarial guard pass rate was 0.0 and the false-convergence detector reported a 66.7% false-convergence rate. This means the graph is too strongly biased toward the invariant and lacks adequate task gating.

Per-Path V0.2 Transfer Performance

Path stack	Trials	Invariant success	Mean guard	Mean risk	Mean final output
A_coordination_logistics	24	66.7%	0.043	0.018	0.946
B_commons_infra_organizing	24	91.7%	0.040	0.021	0.957
C_book_voice_human_surface	24	83.3%	0.030	0.014	0.946
D_exploration_risk_guard	24	79.2%	0.079	0.029	0.945
E_technical_memory_guarded	24	87.5%	0.520	0.008	0.945
F_public_followup	24	83.3%	0.039	0.024	0.952

The weakest transfer success path was A_coordination_logistics at 66.7%. The strongest was B_commons_infra_organizing at 91.7%. The technical_memory_guarded path passed the guard floor and had no guard gaps.

Failure Analysis

Failure	Observed in V0.2	Meaning	Required repair
Adversarial guard failure	Pass rate 0.0	Risk-only test did not suppress final output strongly enough	Make risk-only basin incompatible with final output unless task gate is satisfied
False convergence	66.7%	Negative controls still activated the build-day invariant	Add task-class gates and context-negative controls
Over-convergence	Inferred	Graph has too much attractor pressure toward known invariant	Separate useful convergence from universal convergence

The key failure is not random noise collapse. The key failure is over-convergence. The graph can keep finding the known invariant even when the test context should not be eligible for that invariant. This is a repairable design problem, but it blocks production claims.

Cost-Benefit Assessment

Benefit	Value	Evidence
Improved framing quality	High	Repeated outputs convert event/grant language into durable-process language
Cross-domain transfer	High	Grant invariant transferred into build-day follow-up
Failure-aware learning	High	Guard gaps and false convergence become explicit repair targets
Reproducibility	Medium-high	Harness files and run logs are versionable
Scientific readiness	Medium	Can become paper-ready after more controls and larger runs
Cost/Risk	Severity	Reason
False confidence	High	Internal convergence can be mistaken for truth
Over-convergence	High	V0.2 negative controls activated the invariant too often
Manual overhead	High	Too many records and rules to maintain without tooling
Simulator gap	High	Current harness is local and simplified
Scaling unknown	Medium-high	Millions of tests require batching, logs, thresholds, and data reduction

Million-Test Roadmap

A million-test campaign is plausible, but it should not start until V0.3 fixes false convergence. Otherwise, the system will generate a million confirmations of a biased graph. The next large-scale design should use randomized seeds, task classes, positive controls, negative controls, perturbation sweeps, and strict separation of convergence from verification.

Phase	Scale	Goal	Exit condition
V0.3	1k-10k runs	Repair task gating and false-convergence	False convergence below 5%; guard pass above 80%
V0.4	10k-100k runs	Parameter sweep across noise/dropout/guards	Stable pass region identified
V0.5	100k-1M runs	Monte Carlo robustness	Confidence intervals, failure clusters, reproducible logs
V1.0	Artifact-level tests	Real writing/grant/workflow outputs	Human review plus external evidence where needed

Timing note: the unoptimized Python V0.2 runner completed 192 runs in approximately 1.26 seconds in the current environment. That rough rate would put one million runs on the order of hours if unchanged. This estimate is not a benchmark guarantee; it is only a planning signal. Vectorization, multiprocessing, and reduced logging could improve throughput, while richer scoring could slow it down.

Conclusion

The current evidence supports continuing. The system has a coherent P11 convergence signal and can transfer an invariant across related task types. The V0.2 failure profile is also valuable: the graph is over-convergent and needs task gates before any large-scale claims. The next correct move is not publication yet. The next correct move is V0.3: task-gated negative controls, stronger final-output suppression, and false-convergence repair.